

ObliFlow-GRPO: Training-Free Obligation-Flow Credit Assignment for Long-Horizon Multi-Tool Agents

Yiming Peng^{1*}, Tuo Qu^{1*}, Yuanmeng Zhang¹, Xin Liu^{1,2†}

¹ByteDance

²Institute of Automation, Chinese Academy of Sciences

{pengyiming.001, qutuo.7}@bytedance.com, {zym16784, liuxincasia}@gmail.com

Abstract

Group-based reinforcement learning (RL) is attractive for multi-turn tool agents because it optimizes verifiable outcome rewards without a learned critic, but sparse feedback makes training sample-inefficient: a single outcome must be broadcast over many tool calls and intermediate decisions. A common remedy is finer-grained credit assignment. Recent agentic RL methods instantiate this idea through repeated-state grouping, temporal or state-transition propagation, and learned process scores. These designs localize credit more precisely in time, but their credit units remain state- or score-level proxies rather than the semantic artifacts that later actions consume or transform, such as retrieved pages, selected attributes, normalized values, and execution traces. Consequently, useful artifacts in failed trajectories may be penalized, whereas unused or corrupted artifacts in successful trajectories may receive positive credit. To address this failure mode, which we call *credit-carrier mismatch*, we propose **ObliFlow-GRPO**, a training-free credit assignment framework that requires no auxiliary reward model, value model, verifier training, or decomposition model. It converts task interfaces into verifier-facing checklist obligations and reconstructs each rollout as an Obligation-Artifact Flow Graph, where actions produce typed artifacts, later actions consume or transform them, and validated flows discharge obligations. The resulting potential yields dense flow rewards, while unused artifacts, rejected handoffs, and failed obligations receive waste, break, and min-cut blame signals. This design enables ObliFlow-GRPO to substantially improve training efficiency while attaining state-of-the-art performance on standard agent benchmarks.

Introduction

Transformer-based pretraining underlies modern large language models (Vaswani et al. 2017; Devlin et al. 2019; Brown et al. 2020), which are increasingly used as interactive agents that combine language reasoning (Shao et al. 2024) with external actions (Liu, Li, and Zhao 2026): search, browsing, code execution, and environment interaction (Yao et al. 2022b; Schick et al. 2023; Shridhar et al. 2020; Yao et al. 2022a). This shift changes post-training: agentic RL must optimize long decision sequences from sparse outcome

feedback, where success may be judged only by a final purchase, completed environment goal, passed test, or correct answer despite much earlier evidence.

This intermediate structure is central to multi-tool agency. Agents produce intermediate artifacts that encode evidence, state, and computations; later actions must consume or transform them without breaking task constraints. In shopping, a product page and selected size must remain aligned with the request. In embodied tasks, an observed object, inventory state, and transformation action must form a valid chain. The same pattern appears in data analysis, code execution, browser automation, and retrieval-augmented reasoning (Guu et al. 2020; Lewis et al. 2020). Thus, progress is not captured by the outcome reward alone; it depends on whether intermediate artifacts remain valid when later actions use them.

Group-based RL methods such as GRPO (Shao et al. 2024) are attractive because they avoid learned critics of traditional RL (Sun et al. 2025; Liu et al. 2025), and enable intra-group comparison with verifiable rewards. Recent agentic extensions refine outcome-level credit through repeated-state grouping, temporal propagation, state-transition graphs, or learned process scores (Feng et al. 2026; Zhu et al. 2026; Cheng et al. 2026). Although these methods provide finer credit, they leave a key question under-specified: *what object should receive credit?* Rollouts, states, goal-distance estimates, and step scores are useful proxies, but may not correspond to the objects that later actions actually use.

We call this failure *credit-carrier mismatch*. In long-horizon multi-tool tasks, a successful trajectory can contain redundant searches, unused evidence, or corrupted transformations, while a failed trajectory can still contain valid intermediate artifacts that later actions ignore, misuse, or transform incorrectly. A state may look closer to completion after a promising page is opened even if the selected option violates a constraint. Similarly, a final answer may cite the right evidence but still be wrong if an intermediate artifact is interpreted or reused incorrectly. Thus even step-level credit is noisy when assigned to trajectories, states, or textual evidence rather than verified artifact handoffs.

Figure 1 illustrates this mismatch on WebShop (Yao et al. 2022a). Trajectory credit cannot separate a useful product page from a wrong option. State-distance credit can reward

*These authors contributed equally.

†Corresponding author.

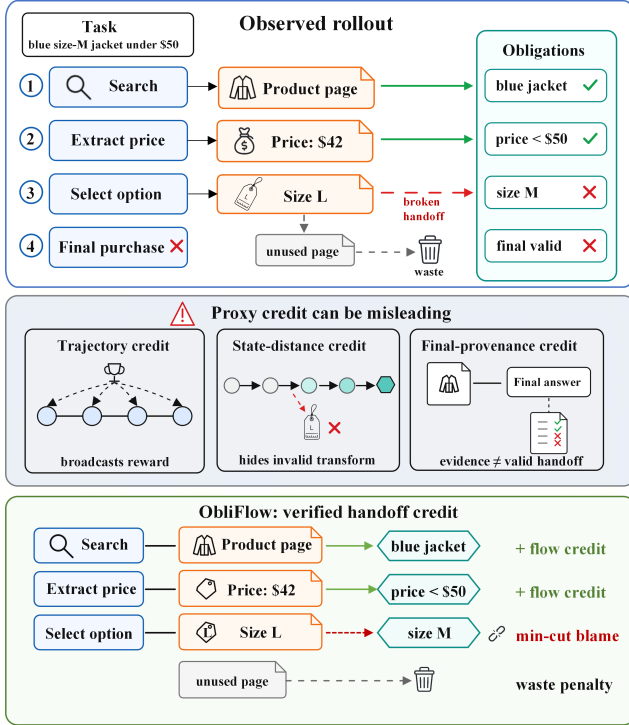


Figure 1: Credit-carrier mismatch in a WebShop rollout. Trajectory credit broadcasts the same outcome signal over the whole rollout, state-distance credit rewards apparent movement toward checkout, and final-provenance credit verifies the cited page, but all can miss the invalid handoff from the correct page to the wrong size option. ObliFlow-GRPO instead rewards valid page and price artifacts while localizing blame to the failed size handoff.

progress toward checkout after the correct page and price are found, but it does not verify that the selected option still satisfies the request. Final-provenance credit can trace the final action to the correct page, yet still miss the invalid handoff from that page to a size-L option when the task requires size M. In each case, the scoring object differs from the process object: progress occurs only when an artifact is produced, stays valid, is consumed or transformed, and discharges a task requirement.

ObliFlow-GRPO addresses this problem by making obligations the bridge between task success and step-level credit. For each task, a frozen LLM-based interface-to-obligation interpreter reads the task instruction, environment verifiers, output schemas, and tool contracts, and converts the success conditions into verifier-facing checklist obligations. The policy LLM then interacts with the environment normally; after each rollout, ObliFlow-GRPO parses its actions, observations, and tool outputs into typed artifacts and links them to obligations through candidate production, consumption, transformation, and discharge edges. Deterministic validators, with a frozen LLM-based semantic verifier only for ambiguous handoffs, accept or reject these edges. Accepted artifact-obligation paths produce flow credit, un-

used artifacts and rejected handoffs receive waste or break penalties, and unsatisfied obligations induce min-cut blame. Thus, the credit construction is training-free: ObliFlow-GRPO trains no process reward model, value model, verifier, or subtask generator, but converts checkable artifact-obligation flow into step-level advantages.

This design gives ObliFlow-GRPO different graph semantics from state-transition approaches. Its nodes are actions, artifacts, and obligations; its edges ask whether information was produced, preserved, consumed, transformed, or discharged under a valid contract. The method can reward useful artifacts inside failed rollouts, penalize unused or corrupted artifacts inside successful rollouts, and localize failed obligations through min-cut blame, improving training efficiency and task performance while preserving critic-free group optimization. The main contributions are:

- **A credit-carrier view of agentic RL.** We identify credit-carrier mismatch in long-horizon multi-tool RL: existing signals often score trajectories, states, or final evidence, while progress is carried by artifacts that must be produced, preserved, transformed, and reused.
- **An obligation-artifact flow formulation.** We cast credit assignment as verified action-artifact-obligation flow, represented by an Obligation-Artifact Flow Graph whose edges validate artifact production, consumption, transformation, and discharge.
- **Training-free flow-based advantages.** We define obligation-flow rewards, waste/break penalties, and min-cut blame, yielding localized step-level advantages without training a process reward model, value model, verifier, or subtask generator.
- **State-of-the-art performance.** We evaluate ObliFlow-GRPO on ALFWorld and WebShop with Qwen2.5-1.5B/7B-Instruct and directly compare against current leading agentic RL baselines, demonstrating stronger overall performance under the same training-step budget.

Related Work

Tool-augmented agents use search, calculation, code execution, retrieval, and environment actions to extend LLM reasoning (Schick et al. 2023; Yao et al. 2022b; Gao et al. 2023; Huang et al. 2022). Programmatic and modular reasoning systems further show that intermediate outputs can be produced and consumed across code or visual modules (Gupta and Kembhavi 2023; Surís, Menon, and Vondrick 2023). Prompting methods such as zero-shot reasoning, self-consistency, and least-to-most prompting improve the quality of reasoning traces (Kojima et al. 2022; Wang et al. 2022; Zhou et al. 2022). Work such as ReAct, Reflexion, and Tree-of-Thoughts improves interaction or deliberation (Yao et al. 2022b; Shinn et al. 2023; Yao et al. 2023), but does not define a training-time credit rule for which tool artifacts were validly handed off.

Interactive benchmarks make this gap concrete. ALFWorld requires agents to discover, move, transform, and place objects across a sequence of observations and actions (Shridhar et al. 2020), while WebShop requires browsing products, comparing attributes, selecting options, and

committing to a purchase under natural-language constraints (Yao et al. 2022a). These environments are often used to evaluate whether an agent eventually succeeds, but their traces also expose a richer structure: intermediate observations, selected entities, tool outputs, and environment states must be preserved across later decisions. Our work focuses on this underused structure as the supervision substrate for credit assignment, rather than on a new prompting strategy or benchmark-specific controller.

Agentic RL methods refine sparse rewards by changing the credit unit. PPO and RLHF optimize policies with learned critics or preference models (Schulman et al. 2017; Stiennon et al. 2020; Ouyang et al. 2022); direct preference optimization avoids explicit reward-model RL in pairwise alignment settings (Rafailov et al. 2023); GRPO removes the value critic with group-relative trajectory advantages (Shao et al. 2024); learned process methods add step-level scores. GiGPO and GAGPO adapt group-based RL to multi-turn agents by repeated-state grouping or temporal propagation (Feng et al. 2026; Zhu et al. 2026). GraphGPO further aggregates rollouts into a state-transition graph and credits transitions that reduce distance to the goal (Cheng et al. 2026). In addition, credit assignment has also been studied in multi-agent RL (Rashid et al. 2020) or embodied RL (Liu et al. 2026), while those methods don’t model LLM tool artifacts as typed carriers. ObliFlow-GRPO uses different graph semantics. Its nodes are actions, artifacts, and obligations, and its edges validate production, consumption, transformation, and discharge. Thus it measures whether information is validly carried to task constraints, not only whether an environment state appears closer to success.

ObliFlow-GRPO is also related to reward shaping and process supervision. Potential-based shaping preserves the outcome objective when the shaping potential is tied to the outcome reward (Ng, Harada, and Russell 1999), but the main design choice is what the potential should measure. Learned process reward models and step-level critics can provide dense feedback, but they introduce additional supervision, model capacity, and failure modes when the learned scorer rewards plausible-looking intermediate text rather than verified downstream use. Step-GRPO is related as a PRM-based step-supervision method for reasoning, but it does not address artifact handoff credit in multi-turn tool environments (Li et al. 2026). Final-evidence or provenance-style analyses can identify which observations appear in an answer, but they are less suited to judging whether an artifact was consumed under the right contract, transformed without semantic loss, or rejected before discharge. ObliFlow-GRPO instead treats credit assignment as a typed flow-verification problem over artifacts and obligations. This keeps the critic-free optimization benefits of GRPO-style training while moving the credit carrier from trajectories, states, or evidence mentions to validated information handoffs.

Methodology

Training setup and credit carrier. For each task prompt q , we sample a rollout group $G(q) = \{\tau_i\}_{i=1}^N$ under the same environment interface, outcome verifier $R(\tau_i)$,

prompt format, sampler, and GRPO-style group comparison as the baselines; ObliFlow-GRPO only changes the carrier of intermediate credit. Rather than broadcasting a trajectory outcome or estimating state-transition progress, it scores whether an action-produced artifact later participates in a verified path to a task obligation. The credit unit is one environment action $a_{i,t}$, so all tokens in the same action span share the same step-level advantage while the task outcome remains trajectory-level. Figure 2 summarizes the pipeline: grouped rollouts instantiate verifier-facing obligations, typed artifacts and handoffs are parsed and validated, flow/cut credit is solved, and the induced per-action advantages are optimized. Because the graph links actions, artifacts, and obligations rather than states, the same rollout batch and outcome reward yield denser action-level advantages with artifact-handoff semantics. Here, training-free means no model parameters are updated for credit construction and no learned reward, value, verifier, or decomposition model is fit on task rollouts; frozen LLM calls, when used, only parse task interfaces or adjudicate ambiguous typed handoffs.

Interface-grounded obligations and ledger. ObliFlow-GRPO augments the outcome reward with verifier-facing obligations $\mathcal{O}_q = \{o_j\}_{j=1}^M$ and typed artifacts $\mathcal{Z}_{i,\leq t}$. Each obligation $o_j = (\ell_j, \tau_j, \psi_j, w_j)$ specifies a canonical role, expected artifact type, checkable predicate, and weight. An interface-to-obligation interpreter derives these obligations from the task instruction, executable success predicates, admissible-action schema, and tool contracts. This interpreter is not a learned planner or human-labeled subtask generator: it exposes the checklist-style conditions already implicit in the task interface. WebShop obligations cover product category, attribute compatibility, option validity, price, and purchase commitment; ALFWorld obligations cover object discovery, possession, state transformation, and final placement.

During rollout, an online ledger $L_{i,t}$ tracks each obligation’s status and the best supporting artifact path observed so far. New observations may bind entities, pages, objects, prices, or states to existing obligations, but monotone ledger updates instantiate interface-implied requirements rather than new success criteria. Thus a useful step advances a verifiable artifact path toward an obligation instead of merely appearing in a successful trajectory. Each entry moves through unresolved, supported, and discharged states: an obligation is supported when an accepted artifact path reaches a compatible predicate and discharged only when the verifier-facing predicate is satisfied. Failed or incompatible handoffs leave the obligation open and become typed blame evidence, so the ledger records support and broken action boundaries rather than planning future actions.

Contract-validated flow graph. Given a rollout, ObliFlow-GRPO constructs an Obligation-Artifact Flow Graph $\mathcal{G}_{i,t} = (\mathcal{A}_{i,\leq t}, \mathcal{Z}_{i,\leq t}, \mathcal{O}_q, \mathcal{E}_{i,\leq t})$. Edges encode four contract-aware relations: production $a_{i,t} \rightarrow z_k$, consumption $z_k \rightarrow a_{i,t}$, transformation $(z_k, a_{i,t}) \rightarrow z_{k'}$, and discharge $z_k \rightarrow o_j$. Candidate edges are considered only when temporal order, type compatibility, and

reward and advantage normalization. Thus failed rollouts can preserve credit for verified useful prefixes, while successful rollouts with unused artifacts or invalid transformations avoid uniformly positive step credit. With $h_{i,t}$ denoting the pre-action history and observation context, the step advantage is assigned to every token in $a_{i,t}$ through a length-normalized action-level ratio,

$$\rho_{i,t}(\theta) = \exp \left(\frac{1}{|a_{i,t}|} \sum_{y \in a_{i,t}} \log \frac{\pi_{\theta}(y \mid h_{i,t}, y_{<})}{\pi_{\text{old}}(y \mid h_{i,t}, y_{<})} \right),$$

$$\mathcal{J}(\theta) = \mathbb{E}_{q,i,t} [\min\{\rho_{i,t}(\theta)A_{i,t}^{\text{ObliFlow}}, \bar{\rho}_{i,t}(\theta)A_{i,t}^{\text{ObliFlow}}\} - \kappa D_{\text{KL}}(\pi_{\theta}(\cdot \mid h_{i,t}) \parallel \pi_{\text{ref}}(\cdot \mid h_{i,t}))], \quad (6)$$

where $\bar{\rho}_{i,t}(\theta) = \text{clip}(\rho_{i,t}(\theta), 1 - \epsilon_c, 1 + \epsilon_c)$. We maximize $\mathcal{J}$ with the same optimizer and KL regularization used by the GRPO/PPO baselines. All added computation is post-rollout graph construction, edge validation, flow solving, and cut-blame assignment; no value model, process reward model, subtask generator, or verifier is trained.

Experiment Settings

Evaluation settings. We evaluate on ALFWorld (Shridhar et al. 2020), whose household tasks require object discovery, transport, transformation, and placement; WebShop (Yao et al. 2022a), whose shopping tasks require search, product inspection, option selection, price/attribute constraints, and purchase; and a controlled multi-tool diagnostic suite with search–fetch–parse–filter–calculate–convert–answer patterns. Together, these settings expose obligations over object identity, possession, state, location, pages, prices, options, confirmations, and labeled intermediate credit carriers.

Comparison set. Head-to-head results focus on GraphGPO (Cheng et al. 2026) and ObliFlow-GRPO. GraphGPO is the closest baseline because it also uses graph-structured credit, but its graph aggregates environment-state transitions. ObliFlow-GRPO instead builds per-rollout action–artifact–obligation graphs that validate consumption, transformation, and discharge. This directly tests whether changing the carrier from states to verified handoffs improves learning under matched rollout and training-step budgets. Other prompting and RL baselines provide context; Table 1 takes all rows except ObliFlow-GRPO from GraphGPO (Cheng et al. 2026).

Interpreter and verifier instrumentation. ObliFlow-GRPO adds two training-free instruments to the standard rollout loop. The *interface-to-obligation interpreter* reads the task instruction, environment verifier, admissible-action schema, and tool contracts to canonicalize verifier-facing obligations without planning or learned decomposition. For WebShop, it creates obligations over category, attribute compatibility, option validity, price, and purchase; for ALFWorld, over object discovery, inventory, transformation, and final placement.

The *contract-aware obligation verifier* validates typed production, consumption, transformation, and discharge

edges between actions, artifacts, and obligations. It uses deterministic checks whenever possible: parsers, exact object-state matches, URL/span matching, schema validation, numerical equivalence, and execution results; a frozen semantic verifier handles only unresolved typed handoffs. The verifier never supplies learned dense rewards; it only accepts, rejects, or leaves unresolved the candidate edges used by the obligation-artifact graph.

Implementation protocol. Text agents use Qwen2.5-1.5B-Instruct and Qwen2.5-7B-Instruct as base LLMs. All training runs use the same environment wrapper and agent-environment interface: prompts contain the instruction, observation, admissible actions, and a fixed recent action–observation window; the model returns a structured environment action after reasoning. Our ObliFlow-GRPO runs match the GraphGPO setting in prompts, memory windows, action schemas, interaction limits, sampling settings, rollout budgets, outcome verifiers, optimizer settings, KL regularization, group sizes, and sampled trajectories per update where applicable. ObliFlow-GRPO reuses collected rollouts: the interpreter initializes obligations once per task prompt, and the verifier checks candidate production, consumption, transformation, and discharge edges after collection. We use Seed1.6 for the interpreter and ambiguous semantic verifier calls because it extracts shallow verifier-facing constraints rather than solving tasks, planning actions, inspecting future success, generating rewards, or acting as a policy. This cost-conscious setting keeps complex tasks reducible to checkable obligations without relying on a frontier model. Deterministic validators are batched and cached by normalized artifact keys, and semantic verifier calls occur only when temporal, type, and ledger filters leave an ambiguous candidate edge. ObliFlow-specific coefficients are fixed across benchmarks; ledger construction, verifier calls, and graph-credit computation are reported separately from rollout and policy-update time.

Experimental Results

Primary task evaluation. Table 1 reports ObliFlow-GRPO alongside the published baseline rows from GraphGPO (Cheng et al. 2026). For ALFWorld, we report success rates for each task category and the overall success rate; for WebShop, we report task score and success rate.

Table 1 shows consistent gains across models and benchmarks. For Qwen2.5-1.5B-Instruct, ObliFlow-GRPO improves over GraphGPO from 92.71 to 95.48 on ALFWorld All, a gain of 2.77 points, and from 78.65 to 80.16 on WebShop success rate, with the WebShop task score also increasing from 89.29 to 91.36. For Qwen2.5-7B-Instruct, the same pattern holds: ObliFlow-GRPO raises ALFWorld All from 95.31 to 96.99, improves WebShop score from 86.94 to 89.07, and improves WebShop success from 80.31 to 81.02. Relative to vanilla GRPO, ALFWorld All improves by 17.62 and 13.66 points for the 1.5B and 7B models, while WebShop success improves by 8.81 and 6.02 points. The benefit is not tied to one model scale or environment: verified handoff credit improves both ALFWorld manipulation and WebShop product selection.

Type	Method	ALFWorld							WebShop	
		Pick	Clean	Cool	Look	Heat	Pick2	All	Score	Succ.
<i>Closed-Source Models</i>										
Prompting	GPT-4o	75.3	60.8	31.2	56.7	21.6	49.8	48.0	31.8	23.7
Prompting	Gemini-2.5-Pro	92.8	63.3	62.1	69.0	26.6	58.7	60.3	42.5	35.9
<i>Qwen2.5-1.5B-Instruct</i>										
Prompting	Qwen2.5	5.9	5.5	3.3	9.7	4.2	0.0	4.1	23.1	5.2
Prompting	ReAct	17.4	20.5	15.7	6.2	7.7	2.0	12.8	40.1	11.3
Prompting	Reflexion	35.3	22.2	21.7	13.6	19.4	3.7	21.8	55.8	21.9
RL Training	PPO	64.8 _(3.5)	40.5 _(6.9)	57.1 _(4.9)	60.6 _(6.6)	46.4 _(4.0)	47.4 _(1.9)	54.4 _(3.1)	73.8 _(3.0)	51.5 _(2.9)
RL Training	RLOO	88.3 _(3.0)	52.8 _(8.6)	71.0 _(5.9)	62.8 _(8.7)	66.4 _(5.5)	56.9 _(4.7)	69.7 _(2.5)	73.9 _(5.6)	52.1 _(6.7)
RL Training	GRPO	82.89 _(3.62)	82.14 _(6.37)	73.86 _(6.84)	78.57 _(0.00)	77.78 _(4.54)	71.43 _(3.89)	77.86 _(1.33)	84.73 _(0.49)	71.35 _(2.05)
RL Training	GiGPO	98.81 _(1.68)	95.16 _(3.89)	81.46 _(0.56)	78.57 _(0.00)	94.44 _(0.00)	93.65 _(5.94)	90.88 _(0.97)	87.94 _(0.43)	73.83 _(2.30)
RL Training	GraphGPO	95.15 _(1.62)	100.0 _(0.00)	85.26 _(2.58)	85.71 _(5.83)	96.30 _(2.61)	93.65 _(2.24)	92.71 _(1.32)	89.29 _(1.48)	78.65 _(3.86)
RL Training	ObliFlow-GRPO	100.0 _(0.00)	100.0 _(0.00)	88.32 _(2.31)	87.49 _(3.92)	98.21 _(1.68)	97.38 _(1.53)	95.48 _(0.83)	91.36 _(0.72)	80.16 _(2.93)
<i>Qwen2.5-7B-Instruct</i>										
Prompting	Qwen2.5	33.4	21.6	19.3	6.9	2.8	3.2	14.8	26.4	7.8
Prompting	ReAct	48.5	35.4	34.3	13.2	18.2	17.6	31.2	46.2	19.5
Prompting	Reflexion	62.0	41.6	44.9	30.9	36.3	23.8	42.7	58.1	28.8
RL Training	PPO	92.3 _(4.0)	64.0 _(8.4)	92.5 _(2.4)	89.5 _(7.0)	80.3 _(2.0)	68.8 _(8.3)	80.4 _(2.7)	81.4 _(3.1)	68.7 _(5.1)
RL Training	RLOO	87.6 _(4.3)	78.2 _(8.3)	87.3 _(5.8)	81.3 _(7.6)	71.9 _(5.2)	48.9 _(8.4)	75.5 _(4.6)	80.3 _(3.2)	65.7 _(4.0)
RL Training	GRPO	88.98 _(5.30)	91.98 _(4.43)	77.89 _(4.58)	78.57 _(0.00)	90.74 _(5.24)	71.43 _(3.89)	83.33 _(2.05)	84.31 _(1.27)	75.00 _(2.78)
RL Training	GiGPO	97.53 _(1.75)	100.0 _(0.00)	83.98 _(1.32)	90.48 _(6.73)	94.44 _(0.00)	100.0 _(0.00)	94.27 _(1.33)	86.72 _(1.44)	78.38 _(1.94)
RL Training	GraphGPO	100.0 _(0.00)	100.0 _(0.00)	91.40 _(1.50)	92.86 _(5.83)	94.44 _(0.00)	92.06 _(2.24)	95.31 _(1.10)	86.94 _(0.68)	80.31 _(1.33)
RL Training	ObliFlow-GRPO	100.0 _(0.00)	100.0 _(0.00)	93.83 _(1.36)	94.20 _(4.27)	96.71 _(1.22)	96.26 _(1.34)	96.99 _(1.02)	89.07 _(0.85)	81.02 _(1.98)

Table 1: Test performance on ALFWorld and WebShop. ALFWorld reports subtask and overall success rates (%); WebShop reports task score and success rate (%). Most results are averaged over 3 random testing seeds, and best performances are highlighted in bold. Baseline rows are taken from GraphGPO (Cheng et al. 2026).

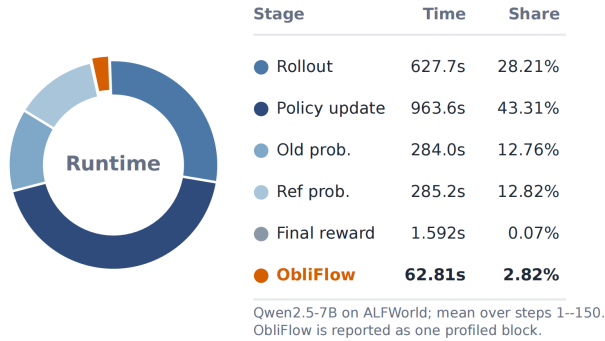


Figure 3: Per-iteration runtime composition. The ObliFlow-GRPO-specific credit construction block accounts for 62.81s, or 2.82% of the total wall-clock time.

Runtime and cost. The added credit construction is lightweight relative to the shared RL loop. Figure 3 profiles Qwen2.5-7B on ALFWorld over training steps 1–150. Rollout, policy update, and old/reference probability recomputation dominate wall-clock time, while the aggregated ObliFlow-GRPO block takes 62.81s per iteration, accounting for only 2.82% of total runtime. Thus ObliFlow-GRPO supplies artifact-flow credit without materially changing the rollout budget or the policy-update cost.

Obligation induction and verification reliability. Because ObliFlow-GRPO relies on interface-derived obli-

gations and verifier-checked discharge edges, we audit both stages: whether induced obligations capture verifier-facing success conditions, and whether the verifier correctly judges discharge by action–artifact flows. Table 2 separates obligation-induction quality from discharge-verification accuracy on ALFWorld and WebShop. For 1,000 held-out prompts from each benchmark, Seed1.6 induces obligations from the instruction, environment verifier, action schema, and tool contracts. A frontier-model expert panel of GPT-5.5, Claude Opus 4.8, and Claude Sonnet 4.6 audits induced obligations and discharge decisions with binary/categorical labels for coverage, correctness, granularity, checkability, and discharge validity. Coverage/correctness ask whether obligations cover verifier-facing success conditions without unsupported constraints; granularity/checkability require atomic, verifiable predicates; discharge validity checks whether an accepted action–artifact path satisfies the target obligation. These frontier models are used only for evaluation, not training or deployment. This is consistent with using Seed1.6 inside ObliFlow-GRPO: the panel provides a high-quality audit signal, while the deployed interpreter extracts shallow checkable constraints. Disagreements are manually reviewed to produce adjudicated verifier-facing labels. Panel agreement measures majority-panel consistency with the final adjudicated labels. Obligation precision and recall are computed at the obligation level against these labels. Discharge accuracy is computed over candidate action–artifact discharge decisions.

Benchmark	Panel Agree.	Obl. Prec.	Obl. Rec.	Discharge Acc.
ALFWorld	95.7	96.8	94.9	97.2
WebShop	94.2	95.1	93.6	95.8

Table 2: Reliability of Seed1.6 obligation induction and discharge verification over 1,000 held-out prompts per benchmark. Panel Agree. reports the frontier panel’s agreement with final adjudicated labels.

Table 2 supports the two lightweight Seed1.6 components used by ObliFlow-GRPO. On ALFWorld, Seed1.6-induced obligations reach 96.8 precision and 94.9 recall against adjudicated verifier-facing labels, while discharge verification reaches 97.2 accuracy. On WebShop, the same interpreter-verifier pipeline remains above 93.6 on every metric, with 95.1 obligation precision, 93.6 obligation recall, and 95.8 discharge accuracy. Because the frontier panel only audits labels and never provides training rewards, Seed1.6 can form checkable obligations and the verifier is reliable enough for graph-credit construction.

Ablation study. Table 3 ablates the components that turn outcome-level task success into localized obligation-flow credit. Removing the outcome-level trajectory advantage tests whether local flow rewards alone are sufficient. Removing consumption or transformation validation tests whether artifacts must be explicitly used and semantically preserved across action boundaries. Removing min-cut blame tests whether failed obligations can be localized to the bottleneck handoff.

Table 3 gives a mechanism-specific readout of why each component is needed. Removing the outcome-level trajectory advantage reduces ALFWorld All from 95.48 to 91.73 and WebShop success from 80.16 to 75.84, confirming that local obligation-flow rewards must remain anchored to actual task completion. Removing consumption validation mostly damages false positive credit: FPC rises from 0.09 to 0.24, showing that unused pages, observations, or tool outputs are incorrectly rewarded when explicit artifact use is not checked. Removing transformation validation instead targets broken handoffs: BHR rises from 0.07 to 0.23, matching the failure mode where an option selection, object-state change, or normalized value corrupts an earlier artifact. Finally, removing min-cut blame leaves aggregate success relatively closer to the full model but sharply degrades failure localization, with Loc.@1 falling from 0.86 to 0.57. Together, the ablation confirms that outcome anchoring, consumption checks, transformation checks, and min-cut blame each repair a different credit-carrier failure mode rather than serving as redundant additions.

Mechanism diagnostics. Final success metrics show task completion, not whether credit reaches the right carriers. A successful trajectory may contain unused artifacts, while a failed one may contain useful artifacts whose handoff later breaks. The controlled diagnostic suite has four families: *Unused-evidence* retrieves a correct artifact that later actions ignore; *Broken-transform* consumes an artifact but corrupts

Variant	ALF All ↑	WS Succ. ↑	FPC ↓	BHR ↓	Loc.@1 ↑
Full	95.48	80.16	0.09	0.07	0.86
no outcome	91.73	75.84	0.13	0.10	0.79
no consume	92.36	76.91	0.24	0.11	0.81
no transform	91.88	76.34	0.14	0.23	0.78
no min-cut	94.21	79.05	0.10	0.08	0.57

Table 3: Main component ablation. Task success and credit-fidelity metrics tie each removed component to a specific credit-carrier failure mode.

Method	FPC ↓	FNC ↓	AWR ↓	BHR ↓
GRPO	0.34	0.29	0.31	0.26
GraphGPO	0.24	0.21	0.25	0.20
Progress RM	0.20	0.18	0.22	0.17
ObliFlow-GRPO	0.08	0.07	0.10	0.06

Table 4: Diagnostic evaluation of credit-carrier mismatch on controlled traces. Lower FPC, FNC, AWR, and BHR indicate fewer false-positive credits, false-negative credits, wasted artifacts, and broken handoffs.

an entity, unit, option, or object state; *Useful-failed-prefix* contains a correct early artifact whose downstream handoff fails; and *State-sparse* perturbs surface text, ads, page order, or observation wording while preserving the obligation. These labels are used only for evaluation and localization, not training.

Each rollout is logged as an obligation-artifact trace with action ids, artifacts, candidate handoffs, validator decisions, obligation states, and per-step advantages. These traces evaluate all methods under the same post-hoc lens, even when a baseline does not train with the ledger. Here FPC, FNC, AWR, and BHR denote false-positive credit, false-negative credit, artifact-waste rate, and broken-handoff rate, respectively. Table 4 shows that ObliFlow-GRPO reduces all four credit-carrier errors relative to GraphGPO and process-reward baselines.

State-sparse robustness against GraphGPO. Following the raw-text versus embedding-based matching diagnostic in GraphGPO (Cheng et al. 2026), we test WebShop perturbations that preserve task constraints but alter observation surfaces through ads, shuffled results, and paraphrases. This diagnostic includes both raw-state and embedding-state GraphGPO variants. Embedding matching reduces state fragmentation but still credits state transitions; ObliFlow-GRPO instead checks whether product pages, options, prices, and purchase actions form valid artifact handoffs. Lower robustness drop, false positive credit, and broken handoff rate therefore test whether artifact-obligation credit differs from stronger state matching. Drop is the clean success rate minus the mean success rate under Ads, Shuffle, and Paraphrase perturbations. Table 5 shows that ObliFlow-GRPO has the smallest robustness drop and the lowest FPC/BHR, indicating that artifact-obligation credit is less sensitive to surface state changes than GraphGPO state matching.

Method	Clean $\uparrow$	Ads $\uparrow$	Shuf. $\uparrow$	Para. $\uparrow$	Drop $\downarrow$	FPC $\downarrow$	BHR $\downarrow$
GRPO	71.35	69.84	68.91	69.26	2.01	0.31	0.28
GiGPO	73.83	71.42	70.58	71.03	2.82	0.27	0.24
GraphGPO-R	78.65	74.22	72.86	73.41	5.15	0.23	0.21
GraphGPO-E	78.65	77.42	76.18	76.72	1.88	0.20	0.18
ObliFlow-GRPO	80.16	79.48	78.91	79.27	0.94	0.09	0.07

Table 5: State-sparse WebShop robustness diagnostic. GraphGPO-R/E denote raw-state and embedding-state variants; Shuf. and Para. denote shuffled search results and paraphrased pages. Drop is clean success minus mean perturbed success; lower FPC/BHR indicates fewer false-positive credits and broken handoffs.

Conclusion

We presented ObliFlow-GRPO, a training-free credit assignment framework for long-horizon multi-tool agents. The paper identifies credit-carrier mismatch: signals based on trajectories, environment states, or final evidence may score proxies for progress, while real task completion depends on verified artifacts that are produced, consumed, transformed, and used to discharge task obligations. ObliFlow-GRPO addresses this mismatch by constructing an Obligation-Artifact Flow Graph, validating typed artifact handoffs, and converting obligation flow, waste, break, and min-cut blame into critic-free step advantages. Experiments on ALFWorld and WebShop show stronger final success rates and diagnostic fidelity than GraphGPO on the reported settings. Learning-curve results are reported in the appendix. Diagnostic metrics in the component ablation further suggest that verified artifact-obligation flow reduces false positive credit, broken handoffs, and failure-localization errors. More broadly, our results suggest that agentic RL benefits from assigning credit to the objects that carry progress across action boundaries, rather than only to trajectories or environment states. Our current evaluation focuses on discrete text-action environments with explicit verifiers and action schemas; extending obligation induction to open-ended, multimodal, or weakly verifiable settings remains future work.

References

Brown, T.; Mann, B.; Ryder, N.; Subbiah, M.; Kaplan, J. D.; Dhariwal, P.; Neelakantan, A.; Shyam, P.; Sastry, G.; Askell, A.; et al. 2020. Language models are few-shot learners. *Advances in neural information processing systems*, 33: 1877–1901.

Cheng, X.; He, S.; Feng, L.; Xu, H.; Yan, M.; Feng, L.; and An, B. 2026. Beyond Trajectory-Level Attribution: Graph-Based Credit Assignment for Agentic Reinforcement Learning. *arXiv preprint arXiv:2605.26684*.

Devlin, J.; Chang, M.-W.; Lee, K.; and Toutanova, K. 2019. Bert: Pre-training of deep bidirectional transformers for language understanding. In *Proceedings of the 2019 conference of the North American chapter of the association for computational linguistics: human language technologies, volume 1 (long and short papers)*, 4171–4186.

Feng, L.; Xue, Z.; Liu, T.; and An, B. 2026. Group-in-group policy optimization for llm agent training. *Advances in Neural Information Processing Systems*, 38: 46375–46408.

Gao, L.; Madaan, A.; Zhou, S.; Alon, U.; Liu, P.; Yang, Y.; Callan, J.; and Neubig, G. 2023. Pal: Program-aided language models. In *International conference on machine learning*, 10764–10799. PMLR.

Gupta, T.; and Kembhavi, A. 2023. Visual programming: Compositional visual reasoning without training. In *Proceedings of the IEEE/CVF conference on computer vision and pattern recognition*, 14953–14962.

Guu, K.; Lee, K.; Tung, Z.; Pasupat, P.; and Chang, M. 2020. Retrieval augmented language model pre-training. In *International conference on machine learning*, 3929–3938. PMLR.

Huang, W.; Abbeel, P.; Pathak, D.; and Mordatch, I. 2022. Language models as zero-shot planners: Extracting actionable knowledge for embodied agents. In *International conference on machine learning*, 9118–9147. PMLR.

Kojima, T.; Gu, S. S.; Reid, M.; Matsuo, Y.; and Iwasawa, Y. 2022. Large language models are zero-shot reasoners. *Advances in neural information processing systems*, 35: 22199–22213.

Lewis, P.; Perez, E.; Piktus, A.; Petroni, F.; Karpukhin, V.; Goyal, N.; Küttler, H.; Lewis, M.; Yih, W.-t.; Rocktäschel, T.; et al. 2020. Retrieval-augmented generation for knowledge-intensive nlp tasks. *Advances in neural information processing systems*, 33: 9459–9474.

Li, W.; Wang, J.; Yu, L.-C.; and Zhang, X. 2026. Step-grpo: Enhancing reasoning quality and efficiency via structured prm-based reinforcement learning. In *Proceedings of the AAAI Conference on Artificial Intelligence*, volume 40, 31734–31742.

Liu, X.; Chen, Y.; Chen, G.; Li, H.; and Zhao, D. 2025. Balancing state exploration and skill diversity in unsupervised skill discovery. *IEEE Transactions on Cybernetics*.

Liu, X.; Li, H.; and Zhao, D. 2026. Videos are sample-efficient supervisions: Behavior cloning from videos via latent representations. *Advances in Neural Information Processing Systems*, 38: 157162–157193.

Liu, X.; Li, Y.; Chen, Y.; Qin, Y.; Li, H.; and Zhao, D. 2026. Dual-Granularity Contrastive Reward via Generated Episodic Guidance for Efficient Embodied RL. *arXiv preprint arXiv:2602.12636*.

Ng, A. Y.; Harada, D.; and Russell, S. 1999. Policy invariance under reward transformations: Theory and application to reward shaping. In *ICML*, volume 99, 278–287. Citeseer.

Ouyang, L.; Wu, J.; Jiang, X.; Almeida, D.; Wainwright, C.; Mishkin, P.; Zhang, C.; Agarwal, S.; Slama, K.; Ray, A.; et al. 2022. Training language models to follow instructions with human feedback. *Advances in neural information processing systems*, 35: 27730–27744.

Rafailov, R.; Sharma, A.; Mitchell, E.; Manning, C. D.; Ermon, S.; and Finn, C. 2023. Direct preference optimization: Your language model is secretly a reward model. *Advances in neural information processing systems*, 36: 53728–53741.

- Rashid, T.; Samvelyan, M.; De Witt, C. S.; Farquhar, G.; Foerster, J.; and Whiteson, S. 2020. Monotonic value function factorisation for deep multi-agent reinforcement learning. *Journal of Machine Learning Research*, 21(178): 1–51.
- Schick, T.; Dwivedi-Yu, J.; Dessì, R.; Raileanu, R.; Lomeli, M.; Hambro, E.; Zettlemoyer, L.; Cancedda, N.; and Scialom, T. 2023. Toolformer: Language models can teach themselves to use tools. *Advances in neural information processing systems*, 36: 68539–68551.
- Schulman, J.; Wolski, F.; Dhariwal, P.; Radford, A.; and Klimov, O. 2017. Proximal policy optimization algorithms. *arXiv preprint arXiv:1707.06347*.
- Shao, Z.; Wang, P.; Zhu, Q.; Xu, R.; Song, J.; Bi, X.; Zhang, H.; Zhang, M.; Li, Y.; Wu, Y.; et al. 2024. Deepseekmath: Pushing the limits of mathematical reasoning in open language models. *arXiv preprint arXiv:2402.03300*.
- Shinn, N.; Cassano, F.; Gopinath, A.; Narasimhan, K.; and Yao, S. 2023. Reflexion: Language agents with verbal reinforcement learning. *Advances in neural information processing systems*, 36: 8634–8652.
- Shridhar, M.; Yuan, X.; Côté, M.-A.; Bisk, Y.; Trischler, A.; and Hausknecht, M. 2020. Alfworld: Aligning text and embodied environments for interactive learning. *arXiv preprint arXiv:2010.03768*.
- Stiennon, N.; Ouyang, L.; Wu, J.; Ziegler, D.; Lowe, R.; Voss, C.; Radford, A.; Amodei, D.; and Christiano, P. F. 2020. Learning to summarize with human feedback. *Advances in neural information processing systems*, 33: 3008–3021.
- Sun, J.; Tu, S.; Zhang, Q.; Li, H.; Liu, X.; Chen, Y.; Chen, K.; and Zhao, D. 2025. Unsupervised zero-shot reinforcement learning via dual-value forward-backward representation. In *International Conference on Learning Representations*, volume 2025, 25604–25633.
- Surís, D.; Menon, S.; and Vondrick, C. 2023. Vipergpt: Visual inference via python execution for reasoning. In *Proceedings of the IEEE/CVF international conference on computer vision*, 11888–11898.
- Vaswani, A.; Shazeer, N.; Parmar, N.; Uszkoreit, J.; Jones, L.; Gomez, A. N.; Kaiser, Ł.; and Polosukhin, I. 2017. Attention is all you need. *Advances in neural information processing systems*, 30.
- Wang, X.; Wei, J.; Schuurmans, D.; Le, Q.; Chi, E.; Narang, S.; Chowdhery, A.; and Zhou, D. 2022. Self-consistency improves chain of thought reasoning in language models. *arXiv preprint arXiv:2203.11171*.
- Yao, S.; Chen, H.; Yang, J.; and Narasimhan, K. 2022a. Webshop: Towards scalable real-world web interaction with grounded language agents. *Advances in Neural Information Processing Systems*, 35: 20744–20757.
- Yao, S.; Yu, D.; Zhao, J.; Shafran, I.; Griffiths, T.; Cao, Y.; and Narasimhan, K. 2023. Tree of thoughts: Deliberate problem solving with large language models. *Advances in neural information processing systems*, 36: 11809–11822.
- Yao, S.; Zhao, J.; Yu, D.; Du, N.; Shafran, I.; Narasimhan, K.; and Cao, Y. 2022b. React: Synergizing reasoning and acting in language models. *arXiv preprint arXiv:2210.03629*.
- Zhou, D.; Schärli, N.; Hou, L.; Wei, J.; Scales, N.; Wang, X.; Schuurmans, D.; Cui, C.; Bousquet, O.; Le, Q.; et al. 2022. Least-to-most prompting enables complex reasoning in large language models. *arXiv preprint arXiv:2205.10625*.
- Zhu, S.; Yu, C.; Yang, R.; Liu, Z.; Hu, J.; Chen, Q.; and Zhang, Y. 2026. GAGPO: Generalized Advantage Grouped Policy Optimization. *arXiv preprint arXiv:2605.13217*.